

Problem A. Grand Mex

第 i 次操作选择哪个端点, 就将边 (x_i, y_i) 指向哪个端点, 因此一个操作方案等价于给树上的每条边定向。考虑固定一个点 v , 将与它相邻的边按照输入顺序记为 $e_1, e_2, \dots, e_d$ 。一条边指向 v 时, a_v 增加 1, 否则 a_v 变为 0, 因此最终的 a_v 恰好等于 $e_1, e_2, \dots, e_d$ 中连续指向 v 的最长后缀长度。

注意到最后一次操作一定会把某个点的值变为 0, 因此最终序列中一定存在 0, 答案至少为 1。另一方面, 将树任意选根, 并把每条边从父亲指向儿子, 此时每个点的入度至多为 1, 从而每个 a_v 都不超过 1, 所以 2 一定没有出现。于是答案只可能是 1 或 2, 问题转化为判断能否使所有点的最终值都不等于 1。

令 $\text{in}(v, e)$ 表示边 e 指向点 v 。若 $d = 1$, 则 $a_v = 1$ 当且仅当 $\text{in}(v, e_1)$ 成立, 因此需要满足

$$\neg \text{in}(v, e_1).$$

若 $d \geq 2$, 则 $a_v = 1$ 当且仅当最后一条边 e_d 指向 v , 而倒数第二条边 e_{d-1} 不指向 v 。因此禁止 $a_v = 1$ 等价于加入限制

$$\neg \text{in}(v, e_d) \vee \text{in}(v, e_{d-1}).$$

对第 i 条边建立一个布尔变量 z_i , 规定

$$z_i = 0 \iff (y_i \rightarrow x_i), \quad z_i = 1 \iff (x_i \rightarrow y_i).$$

于是每个点只会产生一个至多包含两个文字的子句, 整个问题就是一个含 $n - 1$ 个变量和 n 个子句的 2-SAT。

若该 2-SAT 有解, 按照得到的赋值决定每条边的方向。此时所有点的最终值都不为 1, 又因为 0 一定出现, 所以答案为 1。若该 2-SAT 无解, 则不存在答案为 1 的方案; 此时任意选根并将所有边从父亲指向儿子, 每个点的最终值都不超过 1, 所以 2 不出现, 而 1 不可能不出现, 因此答案恰好为 2。

使用 Tarjan 算法求强连通分量即可判断 2-SAT 并构造一组赋值。点、边和子句的数量均为 $O(n)$, 因此时间复杂度为 $O(n)$, 空间复杂度为 $O(n)$ 。实现时使用普通 DFS 即可。

Problem B. 骰子塔

核心观察

每个骰子都能独立旋转，因此一个骰子的最优贡献只取决于它的六个面中哪些面露在外面。用一个六位二进制数 s 表示露出面集合，预处理

$$f[s] = \max_{\text{合法朝向}} \sum_{i \in s} \text{face}_i.$$

标准骰子的相对面为 $(1, 6), (2, 5), (3, 4)$ 。枚举三对相对面分配到三个轴的顺序、每个轴的正反方向，并用奇偶性排除镜像，即可恰好枚举 24 种空间旋转。对每种旋转更新全部 64 个集合的 f 值。

计算一根柱子

考虑网格位置 (x, y) ，其高度为 H ，并把最下面的骰子记为第 1 层。第 z 层骰子的六个面是否可见如下：

- 顶面可见当且仅当 $z = H$ ；
- 底面可见当且仅当 $z = 1$ ；
- 某个侧面可见当且仅当对应相邻柱子的高度小于 z 。

若逐层计算，柱高很大时无法通过。注意可见面集合只会在

$$1, 2, H, H+1, h_{x-1,y}+1, h_{x+1,y}+1, h_{x,y-1}+1, h_{x,y+1}+1$$

这些位置发生变化。把落在 $[1, H+1]$ 内的值排序去重为 $q_0 < q_1 < \dots < q_t$ 。在每个区间 $[q_i, q_{i+1})$ 内，可见面集合恒定；只需在 $z = q_i$ 处求出掩码 s ，将

$$(q_{i+1} - q_i) f[s]$$

加入答案。

边界外的柱高视为 0，这样网格外侧也能统一处理。所有贡献必须使用 64 位整数。

正确性说明

预处理枚举了骰子的全部合法空间旋转，所以 $f[s]$ 正是露出面集合为 s 时的最大点数。每个骰子可独立选朝向，故逐骰子的最优值可以直接相加。

对固定柱子，相邻柱高、柱底和柱顶正是可见性改变的全部临界位置；两个相邻临界位置之间的所有骰子具有相同掩码。按区间长度乘以 $f[s]$ 因而既不遗漏也不重复。再对所有柱子求和即得到全塔最优答案。

复杂度

预处理规模为常数。每根柱子只排序至多 8 个断点，因此总时间复杂度为 $O(NM)$ ，空间复杂度为 $O(NM)$ （若流式保存相邻信息也可进一步压缩）。

Problem C. Phi Master

题解

记上界 $V = 10^7$ 。多组测试数据彼此独立，欧拉函数和质数表只需要预处理一次。

对于任意正整数 x, a ，按质因子逐个考虑欧拉函数。若某个质数 p 同时整除 x 和 a ，则在 $\varphi(x)\varphi(a)$ 的基础上需要额外乘上 $\frac{p}{p-1}$ ；否则不需要额外因子。因此

$$\varphi(xa) = \varphi(x)\varphi(a)\frac{g}{\varphi(g)}, \quad g = \gcd(x, a).$$

于是

$$F_x = \varphi(x) \max_{1 \leq i \leq n} \left(\varphi(a_i) \frac{\gcd(x, a_i)}{\varphi(\gcd(x, a_i))} \right).$$

由于对 $d_1 | d$ ，有 $\frac{d_1}{\varphi(d_1)} < \frac{d}{\varphi(d)}$ ，所以可以把 $d = \gcd(x, a_i)$ 放宽成任意同时满足 $d | x$ 且 $d | a_i$ 的数，有

$$F_x = \varphi(x) \max_{d|x} \left(\frac{d}{\varphi(d)} \max_{1 \leq i \leq n, d|a_i} \varphi(a_i) \right).$$

考虑用一次 Dirichlet 前缀 max 求出 $g_x = \max_{1 \leq i \leq n, d|a_i} \varphi(a_i)$ ，然后再用一次 Dirichlet 前缀 max 求出 $f_x = \max_{d|x} \left(\frac{d}{\varphi(d)} g_d \right)$ ，则最终 $F_x = f_x \varphi(x)$ 。

总共时间复杂度 $\mathcal{O}(TV \log \log V)$ ，空间问题只需要复用 g 数组即可。

Problem D. 三色序列

先考虑一个简化的, $q = 1$ 怎么做?

这个时候有很多的 polylog 做法, 各种神秘数据结构还有类似 THUWC2025 D1T2 的做法。不过为了做多次查询, 应该考虑一个比较快的做法。

注意到一个很强的性质: 若存在一个合法串 S , 前面添加 ≥ 3 个字符后面添加 ≥ 3 个字符, 则一定可以扩展到更长的合法串。

证明考虑一个合法串 S , 不妨设出现次数分别为 a, b, c , 且 $a < b < c$, 考虑怎么让这个字符串不能被扩展: 显然左右应该分别放一个 A 一个 B, 不这么放的话一步就扩展结束了。继续分析, 发现两边第二个元素也不能放 C, 否则只选 $AC + S + B$ 或 $AB + S + C$ 也完成扩展了。最后可以推得, 必须是 $AAB + S + BAA$ 的结构, 但是这种形式只要全选就被扩展了。

所以我们只需要检查一个字符串的 $l \in [1, 3]$ 和 $r \in [n - 2, n]$ 的 6 种情况, 枚举另一个端点即可。

这个还是太慢了, 考虑固定左端点之后怎么找到最远的右端点 (固定右端点同理)。

考虑从一个 $[l, r]$ 往里缩。我们定义三种状态: $\{a, a, a\}$, $\{a, a, b\}$ 和 $\{a, b, b\}$, 分别表示出现次数为 $a = a = a$, $a = a < b$ 和 $a < b = b$ 三种情况。

- 若为 $\{a, a, a\}$, 则连续往前跳长度为 3 的小段, 满足 3 个元素互不相同。接下来遇到的长度为 3 的小段, 一定是包含两种颜色或者只包含一种。包含两种的话扣掉这三个即为合法段, 否则的话会进入形如 $\{a - 3, a, a\}$ 的 case, 后文讨论。

- 若为 $\{a, b, b\}$, 找到目前末尾连续的 a , 跳过去。此时一定形如 $\{a - k, b, b\}$ 。若 $k \neq 0$, 则往前再走一步变成 $\{a - k, b - 1, b\}$ 一定合法 (因为 $a < b$)。若 $k = 0$, 只有 $a = b - 1$ 的时候 $\{a - k, b - 1, b\}$ 不会合法。此时再跳一次, 若跳的元素为 $a - k$ 或者 $b - 1$ 则直接合法, 若跳的是 b 本质又进入了 $\{a, a, a\}$ 的情况。根据上文的讨论, 这种情况预处理跳长度为 3 的元素互不相同小段后, 只需要跳 $\mathcal{O}(1)$ 次。

- 若为 $\{a, a, b\}$, 找到目前末尾连续的 b , 跳过去。若跳的过程中经过了 $\{a, a, b - k = a\}$ 的情况, 就又形成了第一种 case。否则, 代表跳完连续的 b 后形成了 $\{a, a, b - k > a\}$, 此时面前一定是一个 a , 跳过去即可形成 $\{a - 1, a, > a\}$, 形成合法段。

因此, 我们只需要预处理某个颜色 i 的最后一次在 $[1, j]$ 内出现的位置, 以及 i 向前缩掉长度为 3 的互不相同小段后跳到什么位置, 即可枚举 ≤ 3 个左端点并 $\mathcal{O}(1)$ 求得最大合法右端点。这些信息都是可以直接扫一遍序列预处理的。对于固定右端点求最小左端点也同理。

时间复杂度 $\mathcal{O}(n + q)$, 空间复杂度 $\mathcal{O}(n)$ 。

Problem E. 影向拔行

显然我们可以将 C 加上某个充分大的二的幂，所以后面我们不考虑可能产生的负数问题。

考虑第 b 位，下令 $B = 2^b$ ，当 i 逐渐增大时， $C - i$ 每次减一，也就是说，第 b 个 bit 形如一个方波：相同值会出现恰好 B 次，然后翻转，周期为 $2B$ 。

我们从左往右对 r 扫描线，对每个 bit 维护 $\text{tail}[b]$ 表示到目前位置最后一个相同 bit 段的长度， $\text{pre}[b]$ 代表在最后一次反转之前，还能与当前段接成合法后缀的最大长度。如果新元素的第 b 位没有翻转，则只需令 $\text{tail}[b] = \text{tail}[b] + 1$ ，否则，只有当过去的一段长度恰好为 B 时，才能继续接上更早的合法部分，否则只能贡献 $\min(\text{tail}[b], B)$ 。此外，当合法后缀产生了一次翻转时，这次翻转还唯一确定了 $C \bmod 2B$ 的值，记为 $\rho[b]$ ，不难发现：

- 当前位为 1（也就是 $0 \rightarrow 1$ 时）， $\rho[b] = (r - \text{tail}[b]) \bmod 2B$ 。- 当前位为 0（也就是 $1 \rightarrow 0$ 时）， $\rho[b] = (r - \text{tail}[b] + B) \bmod 2B$ 。

接下来我们枚举最高的翻转位 h ，当然可能没有任何位翻转，请大家自行讨论一下这种情况。否则，我们已经固定了 $C \bmod 2^{h+1} = \rho[h]$ ，因此 $b < h$ 的合法情况是可以合法计算的，对于 $b > h$ ，由于 h 是最高位，因此不能产生任何翻转，这也是容易计算的。

现在，我们已经获得了每一位上可以延伸出的最大长度，我们直接排序一下，然后对 k 个子集 ‘checkmax’，之后做一遍超级最大值的高维后缀和即可。

复杂度 $\mathcal{O}(nk^2 \log k + 2^k k)$ 。

Problem F. GCD Master

设

$$f(n) = \sum_{i=1}^n \sum_{j=i}^n \sum_{k=i}^j \gcd(i, k) \gcd(j, k) \binom{j}{k}$$

则有

$$f(n) = f(n-1) + \sum_{i=1}^n \sum_{k=i}^n \gcd(i, k) \gcd(n, k) \binom{n}{k}$$

设

$$g(n) = \sum_{i=1}^n \sum_{k=i}^n \gcd(i, k) \gcd(n, k) \binom{n}{k}$$

那么

$$\begin{aligned} g(n) &= \sum_{i=1}^n \sum_{k=i}^n \gcd(i, k) \gcd(n, k) \binom{n}{k} \\ &= \sum_{k=1}^n \binom{n}{k} \gcd(n, k) \sum_{i=1}^k \gcd(i, k) \end{aligned}$$

设 $a(n) = \sum_{i=1}^n \gcd(i, n)$ 。

$$\begin{aligned} a(n) &= \sum_{i=1}^n \gcd(i, n) \\ &= \sum_{i=1}^n \sum_{x|i, x|n} \varphi(x) \\ &= \sum_{x|n} \varphi(x) \sum_{i=1}^n [x | i] \end{aligned}$$

枚举 x ，计算每个 n 的答案。

对于原式，有

$$\begin{aligned} g(n) &= \sum_{k=1}^n \binom{n}{k} \gcd(n, k) a(k) \\ g(n) &= n! \sum_{k=1}^n \frac{a(k)}{k!(n-k)!} \sum_{x|k, x|n} \varphi(x) \\ &= n! \sum_{x|n} \varphi(x) \sum_{x|k} \frac{a(k)}{k!(n-k)!} \end{aligned}$$

枚举 x ，每次取出 x 倍数的下标做 FFT。

复杂度 $O(n \ln n \log n)$ 。

Problem G. multicon

首先 $\otimes_6$ 是按位独立的，所以我们可以只考虑 $n = 1$ 的情况。我们尝试像异或卷积那样，找到矩阵 M, M^{-1} 来做 DFT 和 IDFT。先说结论，若令 $a' = Ma, b' = Mb, c'_i = a'_i \times b'_i, c = M^{-1}c'$ ，则存在

$$M = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & -1 & 0 & 1 & -1 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 & -1 \end{bmatrix}, M^{-1} = \frac{1}{2} \begin{bmatrix} 2 & 2 & 0 & -2 & -2 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & -1 & -1 & 0 & 1 & 1 \\ 0 & -2 & 0 & 2 & 0 & 0 \\ 0 & -1 & 1 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 & 0 & -1 \end{bmatrix}$$

接下来讨论如何构造出 M 。先考虑和 6 互质的 1, 5。那么令

$$a'_1 = a_1 + a_5, a'_5 = a_1 - a_5, b'_1 = b_1 + b_5, b'_5 = b_1 - b_5, c'_1 = a'_1 \times b'_1, c'_5 = a'_5 \times b'_5$$

则有 $c_1 = \frac{1}{2}(c'_1 + c'_5), c_5 = \frac{1}{2}(c'_1 - c'_5)$ 。

接下来尝试加入单个的 3，显然 $\{1, 3, 5\}$ 与 $\times \pmod{6}$ 构成群，于是有 $(a_1 + a_3 + a_5)(b_1 + b_3 + b_5) = c_1 + c_3 + c_5$ 。

再考虑加入 2, 4，发现在 mod3 下 1, 4 等价，2, 5 等价，而 1, 5 已知，于是可以类似计算 1, 5 地计算出 $c_1 + c_4, c_2 + c_5$ 。

最后加入 0，类似 3 地，我们有 $(\sum_i a_i)(\sum_i b_i) = \sum_i c_i$ 。

整理一下即得上方的 M ，对其求逆或者手动解方程即知 M^{-1} ，总时间复杂度 $O(k^n n)$ ，其中 $k = 6$ 。

Problem H. CuteSafari

不变量与可移动位置

一次操作把字符串分成若干个长度至少为 k 的连续段，并同时交换每段的首尾字符。操作只会置换字符，因此 S 与 T 的字符频次必须完全相同。

先处理三个简单情形：

- $k = 1$ ：长度为 2 的段能交换相邻字符，所以任意排列都可达；
- $n < k$ ：无法进行一次合法操作，只有 $S = T$ 可达；
- $k \leq n < 2k$ ：无法切成两段，唯一非平凡操作是交换整个字符串的首尾。因此只需检查 $S = T$ ，或交换 S_0, S_{n-1} 后等于 T 。

以下设 $n \geq 2k$ 。下标

$$1, 2, \dots, k-2 \quad \text{及} \quad n-k+1, n-k+2, \dots, n-2$$

永远不可能成为某一段的端点，所以这些位置的字符始终不变，必须逐位满足 $S_i = T_i$ 。

其余可移动位置按顺序为

$$0, k-1, k, \dots, n-k, n-1,$$

数量记作

$$r = n - 2k + 4.$$

可达置换

把上述 r 个位置压缩编号。选择一个合法的两段切分，等价于同时交换第一段两个端点和第二段两个端点；选择整串为一段，则交换第一个与最后一个可移动位置。

当 $r \geq 5$ 时，这些生成元可以组合出相邻换位，从而生成全部 r 个位置上的排列。因此，在固定位置逐位相等且总字符频次相同后，不再有其他限制，答案为 **Yes**。

只剩 $r = 4$ ，它恰好对应 $n = 2k$ 。此时设四个可移动字符依次为 a, b, c, d 。可用生成元只有

$$(a \ d), \quad (a \ b)(c \ d),$$

它们生成的群只有 8 个元素。直接从 S 的四元组出发，对这两个置换做一次常数规模 BFS（或硬编码枚举八种结果），检查能否得到 T 的四元组即可。字符允许重复，按实际四元组比较即可。

算法流程

1. 统计两个字符串的 26 个字符频次；不同则输出 **No**。
2. 处理 $k = 1$ 、 $n < k$ 、 $n < 2k$ 三个边界情形。
3. 检查两个永远固定的区间；任一位置不同则输出 **No**。
4. 若 $n - 2k + 4 \geq 5$ ，输出 **Yes**；否则枚举四个可移动位置的 8 个可达状态。

复杂度

每组数据只线性扫描字符串，时间复杂度为 $O(n)$ ，额外空间复杂度为 $O(1)$ （字母表与四元组均为常数规模）。所有测试的总长度不超过 10^7 ，可以直接处理。

Problem I. Imperfect Permutation

记 $m = 2^n$ ，令 idx_x 表示 x 在排列 p 中的位置。

对于原编号连续的一棵子树，设其大小为 s 。状态 (q, v) 表示把它放到第 q 个长度为 s 的对齐块时，内部能够得到的最大匹配数为 v 。只保存 $v > 0$ 的状态。

叶子 x 只有状态 $(idx_x, 1)$ 。设左右儿子的状态分别为 L, R ，父块 q 的两个半块为 $2q, 2q + 1$ ，则

$$dp_q = \max\{L_{2q} + R_{2q+1}, R_{2q} + L_{2q+1}\}.$$

两项分别对应当前节点不交换和交换。不存在的状态视为 0。状态按照位置有序，可以使用双指针线性合并。

大小为 s 的子树最多有 s 个非零状态。同一层所有子树的大小之和为 m ，所以每层复杂度为 $O(m)$ ，总时间复杂度为 $O(m \log m)$ ，空间复杂度为 $O(m)$ 。

Problem J. Card Damage

Card Damage

令总牌数 $n = x + y$ 。

$x = 0$ 时答案显然。

我们首先将出牌刻画为一个由蓝牌和红牌组成的序列。不难发现，这个序列可以被分成若干个块加上一段蓝牌后缀，其中每一块都是若干蓝牌 + 一张红牌的形式。

首先我们证明，若 $x > 0$ ，则最后一张牌一定不是红牌。因为若最后一张牌为红牌，则该牌造成的贡献为 0，将其与任意一张蓝牌交换位置，都将使贡献增加，所以最后一张牌一定不是红牌。

由于最后一张牌的颜色对答案没有影响，因此我们将最后那段蓝牌后缀也看作一块，并将最后一张牌替换为红牌。

由于最后一张牌一定不是红牌，所以我们将分成 $m = y + 1$ 块，记第 i 块长度为 k_i ，则第 i 张红牌打出的位置 $p_i = \sum_{j=1}^i k_j$ 。最后总伤害为 $\sum_{i=1}^m k_i(n - p_i)$ 。

假设我们得到了一种方案 $k_1, k_2, \dots, k_m$ 。现在我们进行调整，我们将段 a 中的一张蓝牌移动到段 b ，则伤害变化量为 $\Delta = (k_a - 1)(n - p_a + 1) + (k_b + 1)(n - p_b) + \sum_{i=a+1}^{b-1} k_i(n - p_i + 1) - \sum_{i=a}^b k_i(n - p_i) =$

$$p_a - p_b - 1 + \sum_{i=a}^{b-1} k_i = \sum_{i=1}^a k_i - \sum_{i=1}^b k_i - 1 + \sum_{i=a}^{b-1} k_i = k_b - k_a - 1。$$

因此，只要我们有 $k_b - k_a > 1$ ，即 $k_b > k_a + 1$ ，我们就可以通过调整使得伤害更高。于是， k_i 只可能取 $\lfloor \frac{n}{m} \rfloor$ 或 $\lceil \frac{n}{m} \rceil$ 。

根据上述交换能够证明，任意排列块长为 $\lfloor \frac{n}{m} \rfloor$ 和 $\lceil \frac{n}{m} \rceil$ 的块对答案没有影响。我们假设所有长度为 $\lceil \frac{n}{m} \rceil$ 的块排在前面 $n \bmod m$ 个，剩余 $n - n \bmod m$ 个为块长为 $\lfloor \frac{n}{m} \rfloor$ 。

因此，我们代入 $k_i = \begin{cases} \lceil \frac{n}{m} \rceil & i \leq n \bmod m \\ \lfloor \frac{n}{m} \rfloor & i > n \bmod m \end{cases}$ 以及 $p_i = \sum_{j=1}^i k_j$ 能够得到最大伤害即为：

$$\sum_{i=1}^{n \bmod m} \lceil \frac{n}{m} \rceil (n - i \lceil \frac{n}{m} \rceil) + \sum_{i=n \bmod m + 1}^m \lfloor \frac{n}{m} \rfloor [n - (n \bmod m) \lceil \frac{n}{m} \rceil - (i - n \bmod m) \lfloor \frac{n}{m} \rfloor]$$

令 $d = n \bmod m$ ， $q_1 = \lfloor \frac{n}{m} \rfloor$ ， $q_2 = \lceil \frac{n}{m} \rceil$ ，则答案为：

$$\begin{aligned} & \sum_{i=1}^d q_2(n - iq_2) + \sum_{i=d+1}^m q_1(n - dq_2 - iq_1 + dq_1) \\ &= \sum_{i=1}^d nq_2 - \sum_{i=1}^d iq_2^2 + \sum_{i=d+1}^m nq_1 - \sum_{i=d+1}^m dq_1q_2 - \sum_{i=d+1}^m iq_1^2 + \sum_{i=d+1}^m dq_1^2 \\ &= ndq_2 - \frac{d(d+1)}{2}q_2^2 + (m-d)nq_1 - (m-d)dq_1q_2 - \frac{(m+d+1)(m-d)}{2}q_1^2 + (m-d)dq_1^2 \end{aligned}$$

上述式子可以 $O(1)$ 计算，总时间复杂度 $O(T)$ 。

Problem K. 推箱子

做法

先固定一个前缀 m ，把这 m 个箱子按初始位置 a 从小到大排序，记排序后为 (a_i, b_i) 。

箱子在数轴上不能重叠，所以它们的相对顺序永远不变。

设第 i 个箱子整个过程中到过的最左位置和最右位置分别为 L_i, R_i 。由于最后要回到 a_i ，这个箱子至少要贡献

$$2(R_i - L_i)$$

次移动。

考虑右端点。若排序后第 j 个箱子到达 b_j ，那么所有 $k \geq j$ 的箱子此时都必须在它右边，并且互不重叠，所以

$$R_k \geq b_j + (k - j).$$

同时 $R_k \geq a_k$ 。因此最小的右端点序列满足

$$R_i = \max(a_i, b_i, R_{i-1} + 1),$$

其中 $R_0 = -\infty$ 。

同理，左端点应尽量大，并且满足

$$L_i = \min(a_i, b_i, L_{i+1} - 1),$$

其中 $L_{m+1} = +\infty$ 。

于是答案为

$$2 \sum_{i=1}^m (R_i - L_i).$$

这个下界可以达到：先按从右到左的顺序把所有箱子从 a_i 移到 R_i ，再按从左到右的顺序从 R_i 移到 L_i ，最后按从右到左的顺序从 L_i 移回 a_i 。因为 L_i 、 a_i 、 R_i 都分别严格递增，整个过程不会发生重叠；每个箱子会经过区间 $[L_i, R_i]$ ，其中包含 b_i 。

在线维护

现在要对每个前缀输出答案，也就是不断插入一个新箱子，并按 a 排序维护上述两个序列。

先看右端点。令

$$h_i = \max(a_i, b_i).$$

需要维护序列

$$R_i = \max(h_i, R_{i-1} + 1).$$

用 FHQ Treap 按 a 作为 key 维护当前序列，并在每个节点维护：

- 子树大小；

- 子树内 R_i 的和;
- 子树最后一个 R_i ;
- 子树中 $\max(R_i - i)$, 这里的 i 是子树内的局部下标;
- 区间赋值懒标记: 把一段连续元素设成 $x + 1, x + 2, \dots$ 。

插入一个新元素时, 把 treap 按 key 分成左、右两部分。设左半部分最后一个右端点为 p , 新元素的右端点为

$$x = \max(h, p + 1).$$

对于右半部分的第 j 个旧元素, 如果它原来的 $R_j \geq x + j$, 它和后面的元素都不用改变; 否则它会被推到 $x + j$ 。因为 $R_j - j$ 单调不降, 被影响的元素一定是右半部分的一个前缀。用维护的 $\max(R_i - i)$ 找到第一个不受影响的位置, 将受影响前缀打上等差赋值懒标记即可。

左端点可以反向转成同一个问题。令

$$g_i = \min(a_i, b_i).$$

把序列按 a 从大到小看, 并维护

$$M_i = \max(-g_i, M_{i-1} + 1).$$

那么 M 的和正好等于 $-\sum L_i$ 。所以第二棵 treap 按 key $-a$ 插入, 元素值为 $-\min(a, b)$ 。

最终每个前缀的答案为

$$2(\text{sumUpper} + \text{sumLower}).$$

复杂度

每次插入进行常数次 split/merge、一次查找和一次懒标记, 期望复杂度 $O(\log n)$ 。

总复杂度为

$$O(n \log n)$$

每个测试用例, 内存复杂度 $O(n)$ 。答案需要使用 64 位整数。

Problem L. P2P 这是一个每个点生成一个权值，并往根节点传递的过程。显然，最多 n 轮后，这棵树除根节点以外的所有点在每个回合都会传递一次值到 B_1 ，因此，我们只需要统计 n 轮内每个点对根节点的贡献次数。

我们只需要统计每个点最早把自己的第一回合产生的值传到根节点。记这个最早回合为 t_x 。若 $f_x < x$ ，显然满足 $t_x = t_{f_x} + 1$ （减不减无所谓）。若存在 $f_x > x$ ，则 $t_x = t_{f_x}$ 。

最后只需要判断 $\sum_{x=2}^n (n - t_x) A_x$ 的符号即可。